

DLT Bronze–Silver–Gold Pipeline for NYC Taxi Data

This pipeline implements the required Bronze → Silver → Gold architecture using Delta Live Tables and the sample NYC taxi dataset provided by Databricks.

Steps to create a pipeline-

1. Go to Jobs & Pipelines in the left menu.
2. Click Create Pipeline.
3. Give the pipeline a name.
4. Choose “Start with an empty notebook/file”.
5. Select the folder location where the SQL file will be stored.
6. Select the language (SQL) for the pipeline file.
7. Enter the DLT transformation code (Bronze → Silver → Gold).
8. Click Run to start the pipeline.
9. After the run completes, the DLT lineage graph appears on the right side.

The screenshot shows the Databricks Lakeflow Pipelines Editor interface. On the left, the SQL code for the pipeline is displayed, defining a Bronze table, a Silver table, and a Gold table. On the right, the Pipeline graph visualizes the data flow between these tables.

```
1 -- BRONZE: Streaming ingestion from sample taxi dataset
2 CREATE OR REFRESH STREAMING TABLE taxi_raw_records
3 (
4   CONSTRAINT valid_distance EXPECT (trip_distance > 0) ON
5   VIOLATION DROP ROW
6 )
7 -- Drops rows where trip_distance <= 0
8 AS
9 SELECT *
10 FROM STREAM(samples.nyctaxi.trips);
11 -- SILVER 1: Suspicious ride detection
12 -- Flags rides where fare is unusually high
13 CREATE OR REFRESH STREAMING TABLE flagged_rides
14 AS
15 SELECT
16   date_trunc("week", tpep_pickup_datetime) AS week,
17   pickup_zip AS zip,
18   fare_amount,
19   trip_distance
20 FROM STREAM(LIVE.taxi_raw_records)
```

The Pipeline graph shows the following structure:

- Streaming table:** `taxi_raw_records` (2s, 1 expectation)
- Materialized view:** `flagged_rides` (1s, 1 expectation)
- Materialized view:** `weekly_stats` (1s, 10 expectations)
- Materialized view:** `top_n` (1s, 3 expectations)

The screenshot shows the Databricks Lakeflow Pipelines Editor interface after the pipeline has been executed. The Pipeline graph on the right shows the lineage of the data. Below the graph, a table catalog is displayed, listing the tables and their properties.

Name	Catalog	Schema	Type	Durati...	Outpu...	Expectations	Dropp...	Warni...	Failed	Incrementaliza...
flagged_rides	misha_11	default	Streaming table	1s	-	Not defined	-	-	-	N/A
taxi_raw_records	misha_11	default	Streaming table	2s	-	1 defined	-	-	0	N/A
top_n	misha_11	default	Materialized view	1s	3	Not defined	-	-	-	No change
weekly_stats	misha_11	default	Materialized view	1s	10	Not defined	-	-	-	No change

1. Ingest raw events into a streaming Bronze table.

2. Apply a DLT expectation to drop records with non-positive trip_distance.

- In the Bronze layer, I ingested raw taxi trip data as a streaming table from samples.nyctaxi.trips.
- A data quality rule was added to drop any records where trip_distance is not positive. This ensures that obviously incorrect or incomplete trips do not move forward in the pipeline.

The screenshot displays the Databricks workspace interface. At the top, the header shows 'Microsoft Azure', 'databricks', a search bar, and user information 'misha_11'. Below the header, a 'New Pipeline 2025-11-23 17:40' is active. The left sidebar shows a file explorer with 'my_transformation.sql' and a 'Run file' button. The main area is split into two panes. The left pane shows a SQL query in a text editor:

```
1 CREATE OR REFRESH STREAMING TABLE taxi_raw_records
2 (
3   CONSTRAINT valid_distance EXPECT (trip_distance > 0) ON VIOLATION DROP ROW
4 )
5 AS
6 SELECT *
```

The right pane shows a 'Pipeline graph' with three nodes: 'Streaming table', 'Materialized view', and 'Materialized view'. The 'Streaming table' node is connected to the 'Materialized view' node. The 'Materialized view' node is connected to the 'Materialized view' node. Below the pipeline graph, a table view for 'taxi_raw_records' is shown. The table has columns: 'tpep_pickup_datetime', 'tpep_dropoff_datetime', '1.2 trip_distance', '1.2 fare_amount', 'pickup_zip', and 'dropoff_zip'. The table contains 1000+ rows of data, with the first 8 rows visible:

	tpep_pickup_datetime	tpep_dropoff_datetime	1.2 trip_distance	1.2 fare_amount	pickup_zip	dropoff_zip
1	2016-02-13T21:47:53.000+00:00	2016-02-13T21:57:15.000+00:00	1.4	8	10103	10110
2	2016-02-13T18:29:09.000+00:00	2016-02-13T18:37:23.000+00:00	1.31	7.5	10023	10023
3	2016-02-06T19:40:58.000+00:00	2016-02-06T19:52:32.000+00:00	1.8	9.5	10001	10018
4	2016-02-12T19:06:43.000+00:00	2016-02-12T19:20:54.000+00:00	2.3	11.5	10044	10111
5	2016-02-23T10:27:56.000+00:00	2016-02-23T10:58:33.000+00:00	2.6	18.5	10199	10022
6	2016-02-13T00:41:43.000+00:00	2016-02-13T00:46:52.000+00:00	1.4	6.5	10023	10069
7	2016-02-18T23:49:53.000+00:00	2016-02-19T00:12:53.000+00:00	10.4	31	11371	10003
8	2016-02-18T20:21:45.000+00:00	2016-02-18T20:38:23.000+00:00	10.15	28.5	11371	11201

At the bottom of the table view, it says '1000+ rows · Truncated data · [Query performance](#)'.

3. Build a Silver table that flags suspicious rides (e.g., fare >> distance) and another Silver table with weekly aggregates.

- The Silver layer contains two tables.
The first Silver table identifies suspicious or unusual rides based on simple business rules.
- A ride is flagged as suspicious if the pickup and dropoff zip codes are the same but the fare is unusually high, or if the trip distance is very short but the fare is still above a threshold.
- The second Silver table calculates weekly statistics, including average fare and average trip distance. These weekly aggregates provide a useful summary of ride trends over time.

The screenshot shows the Databricks interface with a new pipeline created on 2025-11-23 at 17:40. The pipeline graph shows a streaming table 'taxi_raw_records' feeding into a 'flagged_rides' table, which then feeds into a 'weekly_stats' table. The 'flagged_rides' table is currently selected, showing a data table view with 19 rows. The table has columns: week, zip, 1.2 fare_amount, and 1.2 trip_distance. The data shows various rides with different zip codes and fare/distance values.

week	zip	1.2 fare_amount	1.2 trip_distance	
1	2016-01-25T00:00:00.000+00:00	11109	52	3
2	2016-02-15T00:00:00.000+00:00	7311	60	2
3	2016-01-25T00:00:00.000+00:00	10065	52	1.12
4	2016-02-08T00:00:00.000+00:00	11422	52	0.2
5	2016-01-11T00:00:00.000+00:00	11422	52	8.7
6	2015-12-28T00:00:00.000+00:00	10023	52	0.3
7	2016-01-18T00:00:00.000+00:00	10020	52	0.1
8	2016-02-22T00:00:00.000+00:00	10115	55	0.18

The screenshot shows the Databricks interface with the same pipeline. The 'weekly_stats' table is now selected, showing a data table view with 10 rows. The table has columns: week, 1.2 avg_amount, and 1.2 avg_distance. The data shows weekly averages for fare and distance over time.

week	1.2 avg_amount	1.2 avg_distance	
1	2016-02-01T00:00:00.000+00:00	11.990339116719243	2.7463367507886396
2	2016-01-18T00:00:00.000+00:00	11.966793403573064	2.7421759047182723
3	2016-02-15T00:00:00.000+00:00	12.244146522870956	2.8944923763480825
4	2016-01-04T00:00:00.000+00:00	11.907765076862436	2.864603862830116
5	2016-02-08T00:00:00.000+00:00	12.20651356238698	2.751081374321874
6	2015-12-28T00:00:00.000+00:00	12.178038379530918	3.1040618336886974
7	2016-02-22T00:00:00.000+00:00	12.79211403184006	2.9734727878563483
8	2016-02-22T00:00:00.000+00:00	12.626626262626263	2.9734727878563483

4. Produce a Gold materialized view showing top-3 highest fare rides with passenger info and expected schema.

- The Gold layer combines the weekly statistics with the suspicious rides to produce a final table that lists the top three highest-fare suspicious trips.

The screenshot displays the Databricks Lakeflow Pipelines Editor interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and user information (misha_11). The main workspace is titled "New Pipeline 2025-11-23 17:40". On the left, a code editor shows a SQL query for creating a streaming table and a materialized view. The query is as follows:

```
1 CREATE OR REFRESH STREAMING TABLE taxi_raw_records
2 (
3   CONSTRAINT valid_distance EXPECT (trip_distance > 0) ON VIOLATION DROP ROW
4 )
5 AS
6 SELECT *
```

The right side of the interface shows a "Pipeline graph" with nodes for "Streaming table", "Materialized view", and "top_n". Below the graph, a table displays the results of the query. The table has columns for week, 1.2 avg_amount, 1.2 avg_distance, 1.2 fare_amount, 1.2 trip_distance, 1.2 passenger_area, and 1.2 passenger_area. The data is as follows:

week	1.2 avg_amount	1.2 avg_distance	1.2 fare_amount	1.2 trip_distance	1.2 passenger_area	1.2 passenger_area
1 2016-01-04T00:00:00.000+00:00	11.91	2.865	95	5.2	10009	
2 2016-02-15T00:00:00.000+00:00	12.24	2.894	60	2	7311	
3 2016-02-22T00:00:00.000+00:00	12.79	2.973	60	0.92	11422	

The bottom of the interface shows the "Data" tab with a table of results. The table has columns for week, 1.2 avg_amount, 1.2 avg_distance, 1.2 fare_amount, 1.2 trip_distance, 1.2 passenger_area, and 1.2 passenger_area. The data is as follows:

week	1.2 avg_amount	1.2 avg_distance	1.2 fare_amount	1.2 trip_distance	1.2 passenger_area	1.2 passenger_area
1 2016-01-04T00:00:00.000+00:00	11.91	2.865	95	5.2	10009	
2 2016-02-15T00:00:00.000+00:00	12.24	2.894	60	2	7311	
3 2016-02-22T00:00:00.000+00:00	12.79	2.973	60	0.92	11422	

5. Provide a short notebook showing sample queries and validation that the pipeline produces correct results and lineage is visible

- A separate validation notebook was also created to run simple SQL queries on the Bronze, Silver, and Gold tables, confirming that the pipeline produced the correct outputs.